This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors

Riku Murai* Eric Dexheimer* Andrew J. Davison
Imperial College London
{riku.murai15, e.dexheimer21, a.davison}@imperial.ac.uk

Abstract

We present a real-time monocular dense SLAM system designed bottom-up from MASt3R, a two-view 3D reconstruction and matching prior. Equipped with this strong prior, our system is robust on in-the-wild video sequences despite making no assumption on a fixed or parametric camera model beyond a unique camera centre. We introduce efficient methods for pointmap matching, camera tracking and local fusion, graph construction and loop closure, and second-order global optimisation. With known calibration, a simple modification to the system achieves state-of-the-art performance across various benchmarks. Altogether, we propose a plug-and-play monocular SLAM system capable of producing globally consistent poses and dense geometry while operating at 15 FPS.

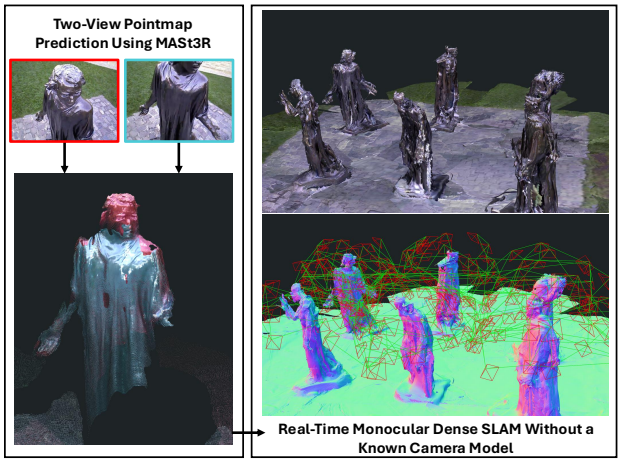


Figure 1. Reconstruction from our dense monocular SLAM system on the Burghers sequence [56]. Using two-view predictions from MASt3R shown on the left, our system achieves globally consistent poses and geometry in real-time without a known camera model.

1. Introduction

Visual simultaneous localisation and mapping (SLAM) is a foundational building block for today's robotics and augmented reality products. With careful design of an integrated hardware and software stack, robust and accurate visual SLAM is now possible. However, SLAM is not yet a plug-and-play algorithm as it requires hardware expertise and calibration. For a minimal single camera setup without additional sensing such as an IMU, in-the-wild SLAM that provides both accurate poses and consistent dense maps does not exist. Achieving such a reliable dense SLAM system would open new research avenues for spatial intelligence.

Performing dense SLAM from only 2D images requires reasoning over time-varying poses and camera models, as well as 3D scene geometry. To solve such an inverse problem of large dimensionality, a variety of priors, from handcrafted to data-driven, have been proposed. Single-view priors, such as monocular depth and normals, attempt to predict geometry from a single image, but these contain ambiguities and lack consistency across views. While multi-view priors like optical flow reduce ambiguity, decoupling pose and geometry is challenging since pixel motion depends on both the extrinsics and the camera model. Although these underlying causes may vary across time and different observers, the 3D scene remains invariant across views. Therefore, the unifying prior required to solve for poses, camera models, and dense geometry from images is over the space of 3D geometry in a common coordinate frame.

Recently, *two-view 3D reconstruction priors*, pioneered by DUSt3R [50] and its successor MASt3R [21], have created a paradigm shift in structure-from-motion (SfM) by capitalising on curated 3D datasets. These networks output pointmaps directly from two images in a common coordinate frame, such that the aforementioned subproblems are implicitly solved in a joint framework. In the future, these priors will be trained on all varieties of camera models with significant distortion. While 3D priors could take in more views, SfM and SLAM leverage spatial sparsity and avoid redundancy to achieve large-scale consistency. A two-view architecture mirrors two-view geometry as the building block of SfM, and this modularity opens the door for both efficient decision-making and robust consensus in the backend.

*Authors contributed equally to this work.

In this work, we propose the first real-time SLAM framework to leverage two-view 3D reconstruction priors as a unifying foundation for tracking, mapping, and relocalisation as shown in Fig. 1. While previous work has applied these priors to SfM in an offline setting with unordered image collections [10], SLAM receives data incrementally and must maintain real-time operation. This requires new perspectives on low-latency matching, careful map maintenance, and efficient methods for large-scale optimisation. Furthermore, inspired by both filtering and optimisation techniques in SLAM, we perform local filtering of pointmaps in the frontend to enable large-scale global optimisation in the backend. Our system makes no assumption on each image's camera model beyond having a unique camera centre that all rays pass through. This results in a real-time dense monocular SLAM system capable of reconstructing scenes with generic, time-varying camera models. Given calibration, we also demonstrate state-of-the-art performance in trajectory accuracy and dense geometry estimation.

In summary, our contributions are:

- The first real-time SLAM system using the two-view 3D reconstruction prior MASt3R [21] as a foundation.
- Efficient techniques for pointmap matching, tracking and local fusion, graph construction and loop closure, and second-order global optimisation.
- A state-of-the-art dense SLAM system capable of handling generic, time-varying camera models.

2. Related Work

To obtain accurate pose estimation, **sparse monocular SLAM** focuses on jointly solving for camera poses and a select number of unbiased 3D landmarks [7]. Algorithmic advances leveraging the sparsity of the optimisation [19] and careful graph construction [26] enabled real-time pose estimation and sparse reconstructions on large scale scenes. While sparse monocular SLAM is very accurate given sufficient features and parallax, it lacks a dense scene model which is useful for both robust tracking and more explicit reasoning over geometry.

To improve robustness and provide interaction, early **dense monocular SLAM** systems demonstrated alternating optimisation of poses and dense depth with handcrafted regularisation [29]. As these systems were limited to controlled settings, recent work has attempted to combine data-driven priors with backend optimisation. While predicting geometric quantities from a single image, such as depth [11, 15, 31, 53] and surface normals [1, 51], have shown significant progress, their use has been limited in SLAM. Predicting geometry from a single-view is ambiguous, resulting in biased and inconsistent 3D geometry. SLAM literature has thus focused on predicting priors over a hypothesis space of possible depths in the form of latent spaces [2, 6], subspaces [41], local primitives [24], and distributions [8, 9]. While the flexibility of these priors can achieve greater consistency, robust correspondence across multiple views is essential.

Multi-view priors, such as multi-view stereo (MVS) [20, 33, 55] and optical flow [43], instead focus on learning correspondence from two or more views as a means to obtaining geometry. However, both require additional information: MVS fixes poses to achieve correspondence, while flow is an entangled observation of motion and geometry subject to the degeneracies mentioned previously. DROID-SLAM [45] combines learned features for matching along with a per-pixel dense bundle adjustment framework into a single end-to-end framework. This results in a robust SLAM system with a backend similar in spirit to sparse SLAM, so the lack of explicit geometric constraints can still produce inconsistent 3D geometry.

Volumetric representations have demonstrated the potential for consistent reconstruction as geometry parameters are coupled in the rendering process. A variety of SLAM systems have adopted differentiable rendering in neural fields [25] and Gaussian splatting [18] for both monocular [23, 58] and RGB-D [16, 39, 52, 57] cameras. However, these methods have lagged in real-time performance compared to alternatives, and require depth, additional 2D priors, or slow camera motion to constrain the solution. 3D priors for general scene reconstruction from images first fuse 2D features into 3D voxel grids which are then decoded into surface geometry [27, 40]. These methods assume known poses for fusion, so are unsuitable for joint tracking and mapping, while the volumetric representations require significant memory and a pre-defined resolution.

All systems mentioned thus far assume known intrinsic calibration. Classical **automatic intrinsic calibration** is possible when there are strict assumptions on scene geometry or unchanging parameters across a set of images [13], but encounters degenerate configurations and sensitivity to noise. Given an initial estimate of intrinsics, refinement via bundle adjustment can improve accuracy online [17], but this already assumes a parametric model and sufficient initialisation of all parameters. Combining DROID-SLAM and self-calibration [12] yields improved robustness to noisy intrinsics, but optimisation is slower due to denser matrix fill-in. More recently, data-driven methods predict intrinsics from one or multiple images [14, 48], but are either limited in accuracy for in-the-wild SLAM or are not flexible in the camera model definition.

Recently, DUSt3R introduced a novel two-view **3D reconstruction prior** that outputs dense 3D point clouds of both images in a *common* coordinate frame. Compared to previously discussed priors that solve subproblems of the task, DUSt3R provides a direct pseudo-measurement of a two-view 3D scene by implicitly reasoning over correspondence, poses, camera models, and dense geometry.

The successor MASt3R [21] predicts additional per-pixel features to improve pixel matching for localisation and SfM [10]. However, as with all priors, predictions can still have inconsistencies and correlated errors in the 3D geometry. DUSt3R and MASt3R-SfM thus require large-scale optimisation for global consistency, but the time complexity does not scale well with the number of images. Spann3R [49] forgoes backend optimisation by fine-tuning DUSt3R to predict a stream of pointmaps directly into a global coordinate system, but must maintain a limited memory of tokens which can cause drift in larger scenes.

In this work, we propose a dense SLAM system built around these two-view 3D reconstruction priors. We only assume a generic central camera model, and propose efficient methods for pointmap matching, tracking and pointmap fusion, loop closure, and global optimisation to achieve large scale consistency of the pairwise predictions in real-time.

3. Method

We provide an overview of the method in Fig. 3, which shows our main components: MASt3R prediction and pointmap matching, tracking and local fusion, loop closure, and global optimisation.

3.1. Preliminaries

DUSt3R takes in a pair of images $\mathcal{I}^i, \mathcal{I}^j \in \mathbb{R}^{H\times W\times 3}$, and outputs pointmaps $\mathbf{X}^i_i, \mathbf{X}^j_i \in \mathbb{R}^{H\times W\times 3}$ along with their confidences $\mathbf{C}^i_i, \mathbf{C}^j_i \in \mathbb{R}^{H\times W\times 1}$. Here, we use notation $\mathbf{X}^i_j$ to express the pointmap of image i represented in the coordinate frame of camera j. In MASt3R, an additional head is added to predict d-dimensional features for matching $\mathbf{D}^i_i, \mathbf{D}^j_i \in \mathbb{R}^{H\times W\times d}$ and its corresponding confidences $\mathbf{Q}^i_i, \mathbf{Q}^j_i \in \mathbb{R}^{H\times W\times 1}$. We define $\mathcal{F}_M(\mathcal{I}^i, \mathcal{I}^j)$ as the forward pass of MASt3R that yields the previously discussed outputs, and throughout the text we will use MASt3R's output directly for conciseness.

While some of the data used to train MASt3R has metric scale, we found that scale is often a large source of inconsistency across predictions. To optimise over differently scaled predictions, we define all poses as $\mathbf{T} \in \mathbf{Sim}(3)$ and updates to the poses using Lie algebra $\boldsymbol{\tau} \in \mathfrak{sim}(3)$ and a left-plus operator:

$$\mathbf{T} = \begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}, \quad \mathbf{T} \leftarrow \boldsymbol{\tau} \oplus \mathbf{T} \triangleq \mathrm{Exp}(\boldsymbol{\tau}) \circ \mathbf{T}, \quad (1)$$

where $\mathbf{R} \in \mathbf{SO}(3)$, $\mathbf{t} \in \mathbb{R}^3$, and scale $s \in \mathbb{R}$, following the notation in [37, 44].

Our only assumption on the camera model is that of a generic central camera [35], which means that all rays pass through a unique camera centre. We define the function $\psi\left(\mathbf{X}^i_i\right)$ that normalises a pointmap $\mathbf{X}^i_i$ into rays of unit norm such that each pointmap defines its own camera model. This enables handling both time-varying camera models, such as zoom, and distortion in a unified manner.

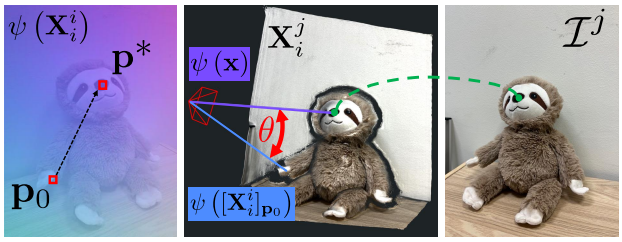


Figure 2. Overview of iterative projective matching: given the two pointmap predictions from MASt3R, the reference pointmap is normalised $\psi\left(\mathbf{X}^i_i\right)$ to give a smooth pixel to ray mapping. For an initial estimate of the projection $\mathbf{p}_0$ of 3D point $\mathbf{x}$ from pointmap $\mathbf{X}^j_i$, the pixel is iteratively updated to minimise the angular difference θ between the queried ray $\psi\left([\mathbf{X}^i_i]_{\mathbf{p}}\right)$ and the target ray $\psi(\mathbf{x})$. After finding the pixel $\mathbf{p}^*$ that achieves the minimum error, we have a pixel correspondence between $\mathcal{I}^i$ and $\mathcal{I}^j$.

3.2. Pointmap Matching

Correspondence is a fundamental component of SLAM that is required for both tracking and mapping. In this case, given the pointmaps and features from MASt3R, we need to find the set of pixel matches between the two images, denoted by $\mathbf{m}_{i,j} = \mathcal{M}(\mathbf{X}^i_i, \mathbf{X}^j_i, \mathbf{D}^i_i, \mathbf{D}^j_i)$. Naive brute-force matching has quadratic complexity since it is a global search over all possible pairs of pixels. To avoid this, DUSt3R uses a k-d tree over 3D points; however, construction is non-trivial to parallelise and the nearest-neighbour search in 3D will find many inaccurate matches if there are errors in the pointmap predictions. In MASt3R, additional high-dimensional features are predicted from the network to achieve wider baseline matching and a coarse-to-fine scheme is proposed to handle the global search. However, the runtime is on the order of seconds for dense pixel matching, and sparse matching is still slower than the k-d tree. Rather than focusing on efficient methods for a global search over matches, we instead find inspiration from optimisation as a local search.

Compared to feature matching, we are motivated by the use of projective data-association commonly used in dense SLAM. However, this requires a parametric camera model with closed-form projection, while our only assumption is that each frame has a unique camera centre. Given the output pointmaps $\mathbf{X}^i_i, \mathbf{X}^j_i$, we can construct the generic camera model of $\mathcal{I}^i$ with the rays $\psi\left(\mathbf{X}^i_i\right)$. Inspired by generic camera calibration methods [32, 35] which lack closed-form projection, we project each point $\mathbf{x} \in \mathbf{X}^j_i$ independently by iteratively optimising the pixel coordinates $\mathbf{p}$ in the reference frame that minimise the ray error:

$$\mathbf{p}^* = \arg\min_{\mathbf{p}} \left\| \psi\left([\mathbf{X}^i_i]_{\mathbf{p}}\right) - \psi(\mathbf{x}) \right\|^2. \quad (2)$$

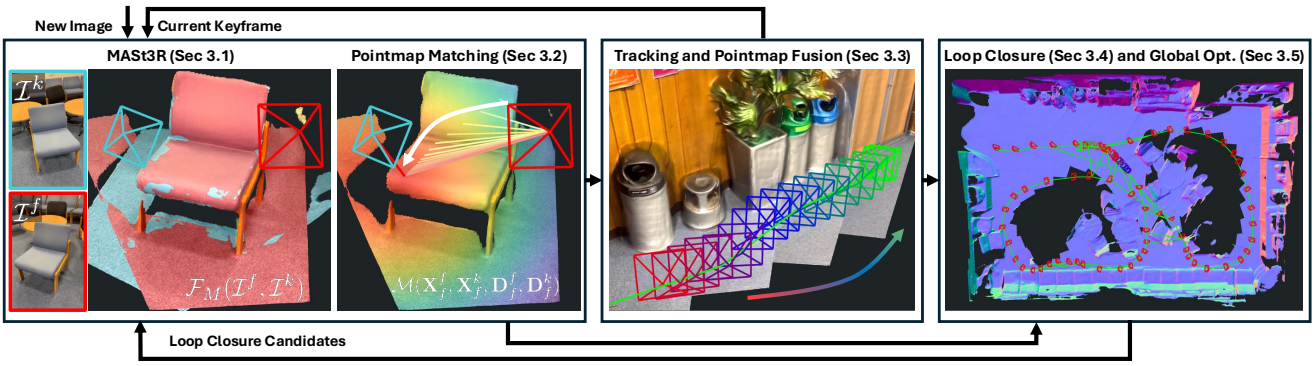


Figure 3. System diagram of MASt3R-SLAM. New images are tracked against the current keyframe by predicting a pointmap from MASt3R and finding pixel matches using our efficient iterative projection pointmap matching. Tracking estimates the current pose and performs local pointmap fusion. When new keyframes are added to the backend, loop closure candidates are selected by querying the retrieval database using encoded MASt3R features. Candidates are then decoded by MASt3R and if a sufficient number of matches is found, edges are added to the backend graph. Large-scale second-order optimisation achieves global consistency of poses and dense geometry.

We show a visual overview in Fig. 2, and note that minimising the Euclidean distance between normalised vectors is equivalent to minimising the angle θ between two normalised rays:

$$\|\psi_1 - \psi_2\|^2 = 2(1 - \cos\theta), \quad \cos\theta = \psi_1^T \psi_2. \tag{3}$$

By using the nonlinear least-squares form similar to [35], we can iteratively solve for updates to projected locations by calculating analytical Jacobians and solving via Levenberg-Marquardt. This can be done separately for each point and converges for almost all valid pixels within 10 iterations as the ray image is smooth. At the end of this process, we now have initial matches $\mathbf{m}_{i,j}$. When there is no initial estimate for the projection $\mathbf{p}$, such as when tracking against a new keyframe or when matching loop closure edges, all pixels are initialised with the identity mapping. During tracking, since we always have the matches from the previous frame, we can use this as initialisation to further speed up the convergence. To handle occlusions and outliers, we also invalidate matches that have large distances in 3D space. Our matching is massively parallel on GPU and additionally can leverage the incremental nature of SLAM.

While these pixels give a good initial estimate of matches using the geometry, MASt3R demonstrates that leveraging per-pixel features greatly improves downstream performance on pose estimation. Since we have a good initialisation from the previous step, we conduct a coarse-to-fine image-based search by updating the pixel location to the maximum feature similarity in a local patch window.

We implement both the iterative projection and feature refinement steps in custom CUDA kernels, as both are parallelisable for each pixel. For tracking this takes only 2 milliseconds and for constructing edges in the graph this takes only a few milliseconds for all newly added edges without any initial estimates of the projections. Note that our matches are unbiased by our pose estimates as they rely purely on the MASt3R outputs, which is atypical for projective data association.

3.3. Tracking and Pointmap Fusion

A key component of SLAM is low-latency tracking of the current frame's pose against the map. As a keyframe-based system, we estimate the relative transformation $\mathbf{T}_{kf}$ between the current frame $\mathcal{I}^f$ and the last keyframe $\mathcal{I}^k$. To be efficient, we would like to use only a single pass of the network to estimate the transformation. Assuming we already have the last keyframe's pointmap estimate $\tilde{\mathbf{X}}_k^k$, we need points in the frame of $\mathcal{I}^f$ to resolve $\mathbf{T}_{kf}$. This can be obtained via $\mathcal{F}_M(\mathcal{I}^f, \mathcal{I}^k)$. One straightforward method to solve for pose is minimising the 3D point error:

$$E_p = \sum_{m,n \in \mathbf{m}_{f,k}} \left\| \frac{\tilde{\mathbf{X}}_{k,n}^k - \mathbf{T}_{kf}\mathbf{X}_{f,m}^f}{w(\mathbf{q}_{m,n}, \sigma_p^2)} \right\|_\rho, \tag{4}$$

where $\mathbf{q}_{m,n} = \sqrt{\mathbf{Q}_{f,m}^f \mathbf{Q}_{f,n}^k}$ is the match confidence weighting proposed in MASt3R-SfM [10]. For robustness, in addition to the Huber norm $\|\cdot\|_\rho$, a per-match weighting is applied:

$$w(\mathbf{q}, \sigma^2) = \begin{cases} \sigma^2/\mathbf{q} & \mathbf{q} > \mathbf{q}_{min} \\ \infty & \text{otherwise} \end{cases}. \tag{5}$$

While $\mathbf{X}_f^k$ instead of $\mathbf{X}_f^f$ could also be aligned to $\mathbf{X}_k^k$ with the benefit of no explicit matching required as they are pixel aligned, we found that explicit matching with $\mathbf{X}_f^f$ had improved accuracy for larger baseline scenarios. More importantly, although the 3D point error is suitable, it is easily skewed by errors in the pointmap predictions as inconsistent predictions in depth are relatively frequent. Since we

ultimately fuse predictions into a single pointmap that averages out all the predictions, error in tracking degrades the keyframe's pointmap that will also be used in the backend.

By again exploiting that the pointmap predictions can be converted to rays under a central camera assumption, we can calculate a directional ray error instead, which is less sensitive to incorrect depth predictions. To calculate this, we simply normalise both points from Eq. (4):

$$E_r = \sum_{m,n \in \mathbf{m}_{f,k}} \left\| \frac{\psi\left(\tilde{\mathbf{X}}^k_{k,n}\right) - \psi\left(\mathbf{T}_{kf}\mathbf{X}^f_{f,m}\right)}{w(\mathbf{q}_{m,n}, \sigma_r^2)} \right\|_\rho . \quad (6)$$

This results in a similar angular error as mentioned in Eq. (3) and shown in Fig. 2, except that we now have many known correspondences and wish to find the pose that minimises all angular errors between canonical rays and corresponding predicted rays from the current frame. Since angular errors are bounded, ray-based errors are robust against outliers [30]. We also include an error term with a small weight on the difference in distances from the camera centre. This prevents the system from becoming degenerate under pure rotation, while avoiding significant bias from errors in depth. We efficiently solve for updates to the pose using Gauss-Newton in an iteratively reweighted least-squares (IRLS) framework. We calculate analytical Jacobians of the ray and distance errors with respect to a perturbation $\boldsymbol{\tau}$ of the relative pose $\mathbf{T}_{kf}$. We stack the residuals, Jacobians, and weights into matrices $\mathbf{r}$, $\mathbf{J}$, and $\mathbf{W}$, respectively. We iteratively solve the linear system and update the pose via:

$$\left(\mathbf{J}^T\mathbf{W}\mathbf{J}\right)\boldsymbol{\tau} = -\mathbf{J}^T\mathbf{W}\mathbf{r}, \quad \mathbf{T}_{kf} \leftarrow \boldsymbol{\tau} \oplus \mathbf{T}_{kf}. \quad (7)$$

Since each pointmap may provide valuable new information, we leverage this by not only filtering over estimates of the geometry, but also over the camera model itself, since it is defined by the rays. After solving for the relative pose, we can use transform $\mathbf{T}_{kf}$ and update the canonical pointmap $\tilde{\mathbf{X}}^k_k$ via a running weighted average filter [5, 28]:

$$\tilde{\mathbf{X}}^k_k \leftarrow \frac{\tilde{\mathbf{C}}^k_k\tilde{\mathbf{X}}^k_k + \mathbf{C}^k_f\left(\mathbf{T}_{kf}\mathbf{X}^k_f\right)}{\tilde{\mathbf{C}}^k_k + \mathbf{C}^k_f}, \tilde{\mathbf{C}}^k_k \leftarrow \tilde{\mathbf{C}}^k_k + \mathbf{C}^k_f . \quad (8)$$

The pointmap initially has larger errors and less confidence due to only using small baseline frames, but filtering merges information from many viewpoints. We experimented with different ways of updating the canonical pointmap, and found that weighted average was best for maintaining coherence while filtering out noise. Compared to the canonical pointmap in MASt3R-SfM [10], we compute this incrementally and require transformation of the points since an additional network prediction of $\mathbf{X}^k_k$ would slow down tracking. Filtering has a rich history in SLAM, and yields the benefit of leveraging information from all frames without having to explicitly optimise for all camera poses and store all predicted pointmaps from the decoder in the backend.

3.4. Graph Construction and Loop Closure

When tracking, a new keyframe $\mathcal{K}_i$ is added if the number of valid matches or the number of unique keyframe pixels in $\mathbf{m}_{f,k}$ falls below a threshold ω_k. After adding $\mathcal{K}_i$, a bidirectional edge to the previous keyframe $\mathcal{K}_{i-1}$is added to the edge-list $\mathcal{E}$. This constrains the estimated poses sequentially in time; however, drift can still occur. To close both small and large loops, we adapt the Aggregated Selective Match Kernel (ASMK) [46, 47] framework used by MASt3R-SfM [10] for image retrieval from encoded features. While this was previously used in a batch setting where all images are available from the start, we modify it to work incrementally. We query the database with the encoded features of $\mathcal{K}_i$ to obtain the top-K images. Since the codebook only has tens of thousands of centroids, we found that a dense L2 distance calculation was sufficiently fast to quantise the features. If the retrieval scores are above a threshold ω_r, we give these pairs to the MASt3R decoder and add bidirectional edges if the number of matches from Sec. 3.2 is above a threshold ω_l. Lastly, we update the retrieval database by adding the new keyframe's encoded features to the inverted file index.

3.5. Backend Optimisation

Given current estimates of keyframe poses $\mathbf{T}_{WC_i}$ and canonical pointmaps $\tilde{\mathbf{X}}^i_i$ for $\mathcal{K}_i$, the goal of the backend optimisation is to achieve global consistency across all poses and geometry. While previous formulations used first-order optimisation and require rescaling after every iteration [10, 50], we introduce an efficient second-order optimisation scheme that handles the gauge freedom of the problem by fixing the first 7-DoF $\mathbf{Sim}(3)$ pose. We jointly minimise the ray error for all edges $\mathcal{E}$ in the graph:

$$E_g = \sum_{i,j \in \mathcal{E}} \sum_{m,n \in \mathbf{m}_{i,j}} \left\| \frac{\psi\left(\tilde{\mathbf{X}}^i_{i,m}\right) - \psi\left(\mathbf{T}_{ij}\tilde{\mathbf{X}}^j_{j,n}\right)}{w(\mathbf{q}_{m,n}, \sigma_r^2)} \right\|_\rho , \quad (9)$$

where $\mathbf{T}_{ij} = \mathbf{T}^{-1}_{WC_i}\mathbf{T}_{WC_j}$. Given N keyframes, Eq. (9) forms and accumulates 14×14 blocks into the $7N \times 7N$ Hessian. We solve this problem again using Gauss-Newton as in Eq. (7) but with sparse Cholesky decomposition as the system is not dense. Construction of the Hessian is made efficient through the use of analytical Jacobians and parallel reductions all implemented in CUDA. Again, a small error term on consistency in distances is added to avoid degeneracy in the pure-rotation case. At most 10 iterations of Gauss-Newton are performed for every new keyframe and optimisation terminates upon convergence. Second-order information greatly speeds up the global optimisation over the alternatives, and our efficient implementation ensures that it is not the bottleneck in the overall system.

Table 1. Absolute trajectory error (ATE (m)) on TUM RGB-D [38].

		360	desk	desk2	floor	plant	room	rpy	teddy	xyz	avg
Calibrated	**ORB-SLAM3** [4]	X	0.017	0.210	X	0.034	X	X	X	**0.009**	-
	DeepV2D [42]	0.243	0.166	0.379	1.653	0.203	0.246	0.105	0.316	0.064	0.375
	DeepFactors [6]	0.159	0.170	0.253	0.169	0.305	0.364	0.043	0.601	0.035	0.233
	DPV-SLAM [22]	0.112	0.018	0.029	0.057	0.021	0.330	0.030	0.084	0.010	0.076
	DPV-SLAM++ [22]	0.132	0.018	0.029	0.050	0.022	0.096	0.032	0.098	0.010	0.054
	GO-SLAM [54]	0.089	**0.016**	0.028	0.025	0.026	0.052	**0.019**	0.048	0.010	0.035
	DROID-SLAM [45]	0.111	0.018	0.042	**0.021**	**0.016**	**0.049**	0.026	0.048	0.012	0.038
	Ours	**0.049**	**0.016**	**0.024**	0.025	0.020	0.061	0.027	**0.041**	**0.009**	**0.030**
Uncalibrated	**DROID-SLAM*** [45, 48]	0.202	0.032	0.091	0.064	0.045	0.918	0.056	0.045	0.012	0.158
	Ours*	0.070	0.035	0.055	0.056	0.035	0.118	0.041	0.114	0.020	0.060

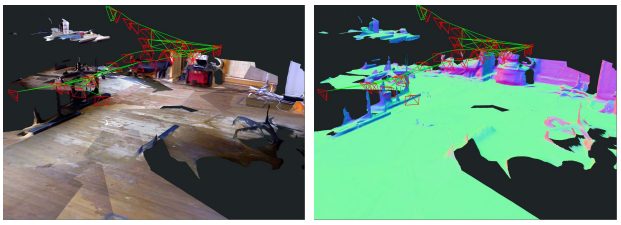


Figure 4. Reconstruction and trajectory TUM fr1/floor sequence.

3.6. Relocalisation

If the system loses tracking due to an insufficient number of matches, relocalisation is triggered. For a new frame, the retrieval database is queried with a stricter threshold on the score. Once the retrieved images have a sufficient number of matches with the current frame, it is then added as a new keyframe into the graph and tracking resumes.

3.7. Known Calibration

Our system works without known camera calibration, but if we do have calibration we can make use of it to improve accuracy via two straightforward changes. First, before canonical pointmaps are used for optimisation in both tracking and mapping, we query only the depth dimension and constrain the pointmap to be backprojected along the rays defined by the known camera model. Second, we change the residuals in optimisation to be in pixel space rather than ray space. In the backend, a pixel $\mathbf{p}^i_{i,m}$ in $\mathcal{I}^i$ is compared against the projection of the 3D point it is matched with:

$$E_{\Pi} = \sum_{i,j \in \mathcal{E}} \sum_{m,n \in \mathbf{m}_{i,j}} \left\| \frac{\mathbf{p}^i_{i,m} - \Pi\left(\mathbf{T}_{ij}\tilde{\mathbf{X}}^j_{j,n}\right)}{w(\mathbf{q}_{m,n}, \sigma^2_{\Pi})} \right\|_{\rho}, \quad (10)$$

where Π is the projection function to pixel space using the given camera model. Furthermore, the additional distance residuals are converted to depth for consistency.

4. Results

We evaluate our system on a wide range of real-world datasets. For localisation, we evaluate monocular SLAM on TUM RGB-D [38], 7-Scenes [36], ETH3D-SLAM [34], and EuRoC [3], all under monocular RGB setting. For geometry evaluation, we use the EuRoC Vicon room sequences as it provides 3D structure scan ground truth, as well as 7-Scenes since it has depth camera measurements.

We run our system on a desktop with Intel Core i9 12900K 3.50GHz and a single NVIDIA GeForce RTX 4090. As our system runs at roughly 15 FPS, we subsample every 2 frames of the datasets to simulate real-time performance. Note that we use the full resolution outputs from MASt3R, which resizes the largest dimension to size 512.

Table 2. Absolute trajectory error (ATE (m)) on 7-Scenes [36].

	chess	fire	heads	office	pumpkin	kitchen	stairs	avg
NICER-SLAM	**0.033**	0.069	0.042	0.108	0.200	**0.039**	0.108	0.086
DROID-SLAM	0.036	0.027	0.025	**0.066**	0.127	0.040	0.026	0.049
Ours	0.053	**0.025**	**0.015**	0.097	**0.088**	0.041	**0.011**	**0.047**
Ours*	0.063	0.046	0.029	0.103	0.114	0.074	0.032	0.066

4.1. Camera Pose Estimation

For all datasets, we report the RMSE of the absolute trajectory error (ATE) in metres. Since all systems are monocular, we perform scaled trajectory alignment. We denote our system without known calibration as Ours*.

TUM RGB-D: On the TUM dataset, we demonstrate state-of-the-art trajectory error when using calibration as shown in Tab. 1. Many of the previously best performing algorithms, such as DROID-SLAM, DPV-SLAM, and GO-SLAM, build on the foundational matching and end-to-end system proposed by DROID-SLAM. In contrast, we propose a unique system that takes an off-the-shelf two-view geometric prior and show that it can outperform other systems while operating in real-time. Furthermore, our uncalibrated system significantly outperforms a baseline, which we denote DROID-SLAM*, that calibrates the intrinsics using GeoCalib [48] on the first image of a sequence, which is then used by DROID-SLAM. We achieve this without assuming a fixed camera model across the entire sequence, and demonstrate the value of 3D priors for dense uncalibrated SLAM over priors that solve subproblems. Our uncalibrated SLAM results are also comparable to results from recent learned techniques such as DPV-SLAM with known calibration.

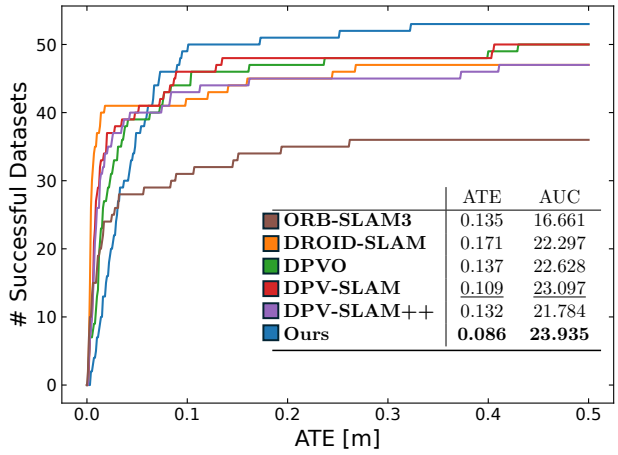


Figure 5. Number of successful trajectories below ATE threshold on ETH3D-SLAM (train) benchmark. The corresponding table shows the mean ATE across completed sequences, as well as the AUC up to the threshold.

7-Scenes: We use the same sequences for evaluation following NICER-SLAM as shown in Tab. 2. Our calibrated system outperforms both NICER-SLAM [58] and DROID-SLAM. Furthermore, our real-time uncalibrated system using a single 3D reconstruction prior outperforms NICER-SLAM, which uses multiple priors in depth, normal, and optical flow networks and runs offline.

ETH3D-SLAM: Due to its difficulty, ETH3D-SLAM has only been evaluated for RGB-D methods. Since the ATE thresholds for the official private evaluation are too strict for monocular methods, we evaluate several state-of-the-art monocular systems on the train sequences and generate the ATE curves. The dataset contains sequences with fast camera motion, hence, for all methods, we do not subsample the frames. While other methods can have more precise trajectories, our method has a longer tail in terms of robustness, resulting in both the best ATE and area-under-curve (AUC).

EuRoC: We report the average ATE across all 11 EuRoC sequences in Tab. 3. For the uncalibrated case, we found that the distortion was too significant as MASt3R was not yet trained on such camera models, so we undistorted the images but did not give calibration to the rest of the pipeline. In general, our system is outperformed by DROID-SLAM, but it explicitly augments its training with 10% greyscale images. However, 0.041m ATE is still very accurate, and from the comparisons in [22], all outperforming methods build on top of the foundation from DROID-SLAM, while we present a novel method using a 3D reconstruction prior.

4.2. Dense Geometry Evaluation

We evaluate our geometry against DROID-SLAM and Spann3R [49] on the EuRoC Vicon room sequences and 7-Scenes seq-01. For EuRoC, the alignment between the reference and the estimated point cloud is obtained by aligning the estimated trajectory against the Vicon trajectory. Note, that this setup favours DROID-SLAM which obtains lower trajectory error. For 7-Scenes, we backproject the depth images using poses provided by the dataset to create the reference point cloud. It is then aligned to the estimated point cloud using ICP as the extrinsic calibration between RGB and depth sensor is not provided.

We report the RMSE for accuracy, which is defined as the distance between each estimated point and its nearest reference point, and completion, the distance between each reference point and its nearest estimated point. Both metrics are calculated with a maximum distance threshold of 0.5m and averaged across all sequences. We also report Chamfer Distance, the average of the two metrics.

Tab. 3 summarises the geometry evaluation on 7-Scenes and EuRoC. For 7-Scenes, both our method with and without calibration and Spann3R achieve more accurate reconstruction compared to DROID-SLAM, highlighting the advantage of the 3D prior. We run Spann3R under two different settings. In one, a keyframe is taken every 20 images and in the other every 2 images. The discrepancy in the two settings shows the challenges test-time optimisation-free approaches face to generalise. Ours without calibration performs the best in both Accuracy and Chamfer distance. This can be attributed to the fact that the intrinsic calibration 7-Scenes provides is the default factory calibration.

For EuRoC, Spann3R struggles as the sequences are not object-centric and thus is excluded. As summarised in Tab. 3, although DROID-SLAM outperforms our method in terms of ATE, our method with/without calibration obtains better

Table 3. Reconstruction Evaluation on 7-Scenes and EuRoC with all metrics in metres.

7-scenes	ATE	Accuracy	Completion	Chamfer
DROID-SLAM	0.049	0.115	**0.040**	0.077
Spann3R @20	N/A	0.069	0.047	0.058
Spann3R @2	N/A	0.124	0.043	0.084
Ours	**0.047**	0.074	0.057	0.066
Ours*	0.066	**0.068**	0.045	**0.056**
EuRoC	ATE	Accuracy	Completion	Chamfer
DROID-SLAM	**0.022**	0.173	**0.061**	0.117
Ours	0.041	**0.099**	0.071	**0.085**
Ours*	0.164	0.108	0.072	0.090

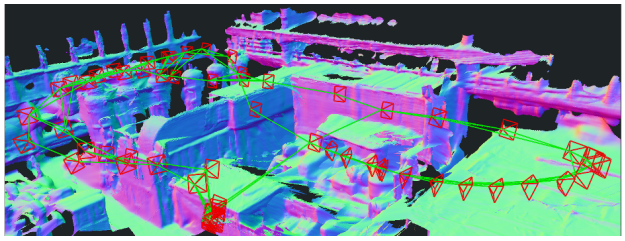


Figure 6. Reconstruction on EuRoC Machine Hall 04.

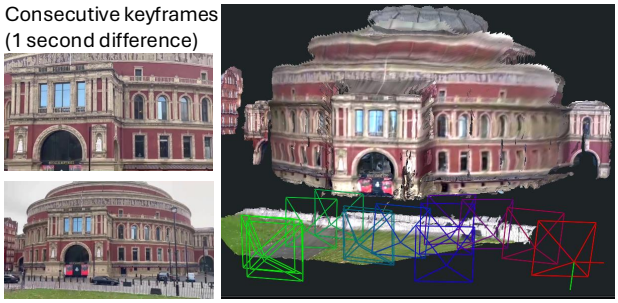


Figure 7. Dense uncalibrated SLAM with extreme zoom changes shown by two consecutive keyframes for an outdoor scene.

geometry. DROID-SLAM obtains higher completion as it estimates a large number of noisy points which surround the reference point cloud, but our method has significantly better accuracy. It is interesting to note that our uncalibrated system has a noticeably larger ATE, but still outperforms DROID-SLAM in Chamfer distance.

4.3. Qualitative Results

Fig. 1 shows a reconstruction of the challenging Burghers sequence which has few matchable features on the specular figures. We show examples of pose estimation and dense reconstructions for TUM in Fig. 4 and for EuRoC in Fig. 6. Furthermore, we show an example with extreme zoom changes between consecutive keyframes in Fig. 7.

4.4. Component Analysis

We compare matching techniques in Tab. 4. Our parallelised projective matching with feature refinement achieves the best accuracy with significantly faster runtime. Performing MASt3R matching over all pixels takes 2 seconds, while our matching takes 2ms and makes the entire system FPS nearly 40x faster. Please refer to the supplementary for a full runtime analysis of the system. In Tab. 5, we test different methods for updating the canonical pointmap and report the average ATE across TUM, 7-Scenes, and EuRoC. Selecting the most recent and first pointmaps incur drift and lack sufficient baseline, respectively. Given calibration, weighted fusion performs on par with selecting the pointmap with the highest median confidence, but it achieves the lowest ATE without calibration and improves the ATE on EuRoC by 1.3cm, indicating that fusing over camera models is important. In Tab. 6, the ray error formulation for uncalibrated tracking and backend optimisation improves performance over using the 3D point error which contains inaccurate depth predictions. Tab. 7 shows that loop closure improves both pose and geometry accuracy, with more significant gains on longer sequences. This demonstrates that the outputs of MASt3R still contain bias and cause drift, which our components are designed to mitigate.

Table 4. Matching comparison.

	ATE [m] w/ calib	ATE [m] w/o calib	Matching Time [ms]	System FPS
k-d tree	0.061	0.115	40	8.8
MASt3R	0.042	0.098	2000	0.4
Ours	0.062	**0.092**	**0.5**	**15.1**
Ours + feat	**0.039**	0.097	2	14.9

Table 5. Fusion methods.

	ATE [m] w/o calib	ATE [m] w/ calib
Recent	0.207	0.160
First	0.114	0.059
Median	0.102	**0.039**
Weighted	**0.097**	**0.039**

Table 6. ATE (m) for error formulation in format (point / ray).

TUM	7-Scenes	EuRoC	avg
0.092 / **0.060**	0.084 / **0.066**	0.290 / **0.164**	0.155 / **0.097**

Table 7. Loop closure ablation in format (without LC / with LC).

	ATE (m)			Chamfer (m)
	TUM	7-scenes	EuRoC Vicon	EuRoC Vicon
Calib	0.064 / **0.030**	0.066 / **0.047**	0.233 / **0.029**	0.151 / **0.085**
No Calib	0.090 / **0.060**	0.075 / **0.066**	0.349 / **0.122**	0.179 / **0.090**

5. Limitations and Future Work

While we can estimate accurate geometry by filtering pointmaps in the frontend, we do not currently refine all geometry in the full global optimisation. While DROID-SLAM optimises per-pixel depth via bundle adjustment, this framework permits incoherent geometry. A method that can make pointmaps globally consistent in 3D while retaining the coherence of the original MASt3R predictions all in *real-time* would be an interesting direction for future work.

Since MASt3R is only trained on images with pinhole images, its geometry predictions degrade with increasing distortion. However, in the future, models will be trained on a variety of camera models and will be compatible with our framework that never assumes a parametric camera model. Furthermore, using the decoder at full resolution is currently a bottleneck, especially for low-latency tracking and checking loop closure candidates. Improving network throughout will benefit the total system efficiency.

6. Conclusion

We present a real-time dense SLAM system based on MASt3R that handles in-the-wild videos and achieves state-of-the-art performance. Much of the recent progress in SLAM has followed the contributions of DROID-SLAM, which trains an end-to-end framework that solves for poses and geometry from a flow update. We take a different approach by building a system around an off-the-shelf geometric prior that achieves comparable pose estimation for the first time, while also providing consistent dense geometry.

7. Acknowledgement

This research is supported by the Engineering and Physical Sciences Research Council [grant number EP/W524323/1].

References

[1] Gwangbin Bae and Andrew J. Davison. Rethinking inductive biases for surface normal estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[2] Michael Bloesch, Jan Czarnowski, Ronald Clark, Stefan Leutenegger, and Andrew J. Davison. CodeSLAM - learning a compact, optimisable representation for dense visual SLAM. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 2

[3] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W. Achtelik, and Roland Siegwart. The EuRoC micro aerial vehicle datasets. *International Journal of Robotics Research (IJRR)*, 35(10), 2016. 6

[4] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM. *IEEE Transactions on Robotics (T-RO)*, 2021. 6

[5] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH*, 1996. 5

[6] Jan Czarnowski, Tristan Laidlow, Ronald Clark, and Andrew J. Davison. DeepFactors: Real-time probabilistic dense monocular SLAM. *IEEE Robotics and Automation Letters (RA-L)*, 2020. 2, 6

[7] Andrew J. Davison, Ian D. Reid, Nicholas D. Molton, and Olivier Stasse. MonoSLAM: Real-time single camera SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI)*, 2007. 2

[8] Eric Dexheimer and Andrew J. Davison. Learning a depth covariance function. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2

[9] Eric Dexheimer and Andrew J. Davison. COMO: Compact mapping and odometry. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 2

[10] Bardienus Duisterhof, Lojze Zust, Philippe Weinzaepfel, Vincent Leroy, Yohann Cabon, and Jerome Revaud. MASt3R-SfM: a fully-integrated solution for unconstrained structure-from-motion. *arXiv preprint arXiv:2409.19152*, 2024. 2, 3, 4, 5

[11] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Neural Information Processing Systems (NeurIPS)*, 2014. 2

[12] Annika Hagemann, Moritz Knorr, and Christoph Stiller. Deep geometry-aware camera self-calibration from video. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2023. 2

[13] Richard Hartley and Andrew Zisserman. *Multiple view geometry in computer vision*. Cambridge university press, 2003. 2

[14] Linyi Jin, Jianming Zhang, Yannick Hold-Geoffroy, Oliver Wang, Kevin Matzen, Matthew Sticha, and David F. Fouhey. Perspective fields for single image camera calibration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2

[15] Bingxin Ke, Anton Obukhov, Shengyu Huang, Nando Metzger, Rodrigo Caye Daudt, and Konrad Schindler. Repurposing diffusion-based image generators for monocular depth estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[16] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[17] Nima Keivan and Gabe Sibley. Constant-time monocular self-calibration. In *2014 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2014. 2

[18] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. 2

[19] Georg Klein and David Murray. Parallel tracking and mapping for small AR workspaces. In *Proceedings of the International Symposium on Mixed and Augmented Reality (ISMAR)*, 2007. 2

[20] Lukas Koestler, Nan Yang, Niclas Zeller, and Daniel Cremers. TANDEM: Tracking and dense mapping in real-time using deep multi-view stereo. In *Conference on Robot Learning (CoRL)*, 2022. 2

[21] Vincent Leroy, Yohann Cabon, and Jerome Revaud. Grounding image matching in 3D with MASt3R. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 1, 2, 3

[22] Lahav Lipson, Zachary Teed, and Jia Deng. Deep patch visual SLAM. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 6, 7

[23] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian splatting SLAM. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[24] Kirill Mazur, Gwangbin Bae, and Andrew J. Davison. SuperPrimitive: Scene reconstruction at a primitive level. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[25] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. 2

[26] Raúl Mur-Artal, J. M. M. Montiel, and Juan D. Tardós. ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics (T-RO)*, 31(5), 2015. 2

[27] Zak Murez, Tarrence van As, James Bartolozzi, Ayan Sinha, Vijay Badrinarayanan, and Andrew Rabinovich. Atlas: End-to-end 3D scene reconstruction from posed images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. 2

[28] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet

Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. KinectFusion: Real-time dense surface mapping and tracking. In *Proceedings of the International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011. 5

[29] Richard. A. Newcombe, Steven J. Lovegrove, and Andrew J. Davison. DTAM: Dense tracking and mapping in real-time. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2011. 2

[30] Linfei Pan, Daniel Barath, Marc Pollefeys, and Johannes Lutz Schönberger. Global structure-from-motion revisited. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 5

[31] Rene Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI)*, 2022. 2

[32] Dennis Rosebrock and Friedrich M. Wahl. Generic camera calibration and modeling using spline surfaces. In *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, 2012. 3

[33] Mohamed Sayed, John Gibson, Jamie Watson, Victor Prisacariu, Michael Firman, and Clément Godard. SimpleRecon: 3D reconstruction without 3D convolutions. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022. 2

[34] Thomas Schöps, Torsten Sattler, and Marc Pollefeys. BAD SLAM: Bundle adjusted direct RGB-D SLAM. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 6

[35] Thomas Schöps, Viktor Larsson, Marc Pollefeys, and Torsten Sattler. Why having 10,000 parameters in your camera model is better than twelve. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3, 4

[36] Jamie Shotton, Ben Glocker, Christopher Zach, Shahram Izadi, Antonio Criminisi, and Andrew Fitzgibbon. Scene coordinate regression forests for camera relocalization in RGB-D images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013. 6

[37] Joan Sola, Jeremie Deray, and Dinesh Atchuthan. A micro lie theory for state estimation in robotics. *arXiv preprint arXiv:1812.01537*, 2018. 3

[38] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *Proceedings of the IEEE/RSJ Conference on Intelligent Robots and Systems (IROS)*, 2012. 6

[39] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J. Davison. iMAP: Implicit mapping and positioning in real-time. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2021. 2

[40] Jiaming Sun, Yiming Xie, Linghao Chen, Xiaowei Zhou, and Hujun Bao. NeuralRecon: Real-time coherent 3D reconstruction from monocular video. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2

[41] Chengzhou Tang and Ping Tan. BA-Net: Dense bundle adjustment networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. 2

[42] Zachary Teed and Jia Deng. DeepV2D: Video to depth with differentiable structure from motion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020. 6

[43] Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. 2

[44] Zachary Teed and Jia Deng. Tangent space backpropagation for 3D transformation groups. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 3

[45] Zachary Teed and Jia Deng. DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras. In *Neural Information Processing Systems (NeurIPS)*, 2021. 2, 6

[46] Giorgos Tolias, Yannis Avrithis, and Hervé Jégou. To aggregate or not to aggregate: Selective match kernels for image search. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2013. 5

[47] Giorgos Tolias, Tomas Jenicek, and Ondřej Chum. Learning and aggregating deep local descriptors for instance-level recognition. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. 5

[48] Alexander Veicht, Paul-Edouard Sarlin, Philipp Lindenberger, and Marc Pollefeys. GeoCalib: Single-image calibration with geometric optimization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 2, 6

[49] Hengyi Wang and Lourdes Agapito. 3D reconstruction with spatial memory. *arXiv preprint arXiv:2408.16061*, 2024. 3, 7

[50] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. DUSt3R: Geometric 3D vision made easy. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1, 5

[51] Xiaolong Wang, David Fouhey, and Abhinav Gupta. Designing deep networks for surface normal estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015. 2

[52] Chi Yan, Delin Qu, Dan Xu, Bin Zhao, Zhigang Wang, Dong Wang, and Xuelong Li. GS-SLAM: Dense visual SLAM with 3D Gaussian splatting. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[53] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[54] Youmin Zhang, Fabio Tosi, Stefano Mattoccia, and Matteo Poggi. GO-SLAM: Global optimization for consistent 3D instant reconstruction. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2023. 6

[55] H. Zhou, B. Ummenhofer, and T. Brox. DeepTAM: Deep tracking and mapping. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018. 2

[56] Qian-Yi Zhou and Vladlen Koltun. Dense scene reconstruction with points of interest. *ACM Transactions on Graphics*, 32(4), 2013. 1

[57] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R. Oswald, and Marc Pollefeys. NICE-SLAM: Neural implicit scalable encoding for SLAM. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2

[58] Zihan Zhu, Songyou Peng, Viktor Larsson, Zhaopeng Cui, Martin R Oswald, Andreas Geiger, and Marc Pollefeys. NICER-SLAM: Neural implicit scene encoding for RGB SLAM. In *Proceedings of the International Conference on 3D Vision (3DV)*, 2024. 2, 7